Huffman Coding

A Greedy Approach to Lossless Data Compression

Souptik Bhattacharjee - Roll: 260
Manab Patra – Roll: 257
Archisman De – Roll 314

What is Data Compression?

Definition

Data Compression is the process of reducing the amount of storage space or number of bits required to represent data, while preserving the information contained in that data.

Relation b/w encoding and Data Compression

Compression algorithms often **use encoding techniques to achieve a more compact representation**.

Why is it needed?

1. Reduce Storage Space.
2. Reduce Transmission Time
3. Reduce Bandwidth Usage
4. Improve Efficiency
5. Reduce Storage and Transmission Cost

What is Encoding?

Definition

Encoding represents characters or symbols using a defined code so computers can store, process, or transmit them. For text, the code is often represented as bits (0 and 1).

Simple example

ASCII: A → 1000001
$A = 65_{10} = 1000001_2$

Why is encoding needed?

- Computers operate on binary data.
- A common code allows different systems to interpret the same data.
- Encoding provides a standard mapping from symbols to codewords.

Types used in this presentation

1. Fixed-Length Encoding
2. Variable-Length Encoding
.

Fixed-Length Encoding

Definition

Every symbol is assigned a codeword with the same number of bits, independent of symbol frequency.

Technique	Typical width	Main purpose
ASCII	7 bits	Standard English text
Extended ASCII / code pages	8 bits	Additional Latin characters
EBCDIC	8 bits	IBM mainframes / legacy systems

ASCII — 7-bit Fixed-Length Encoding

Core facts

ASCII (American Standard Code for Information Interchange) uses 7 bits per character, giving $2^7 = 128$ code points.

Advantages

- Simple and compact for basic English text
- Easy to implement and decode
- Became a foundational character standard

Character	Decimal	Binary
A	65	1000001
B	66	1000010
C	67	1000011
D	68	1000100
E	69	1000101

Limitations

- Only 128 codes
- Cannot represent most world scripts

Extended ASCII — 8-bit Code Pages

What changed?

Using 8 bits gives $2^8 = 256$ possible byte values. Many vendor-specific 'extended ASCII' code pages used the extra range for accented Latin characters, and symbols.

Example symbol	Example code-page use
à	accented Latin
é	accented Latin
ö	accented Latin
£	currency symbol
©	copyright symbol
µ	micro sign

Why was it still insufficient?

It is still useful only for American and West-European Countries.

EBCDIC — IBM's 8-bit Character Code

Background

EBCDIC (Extended Binary Coded Decimal Interchange Code) was developed by IBM and became important in mainframe and enterprise environments.

Character	EBCDIC hex
A	C1
B	C2
C	C3

Advantages

- Well integrated with IBM mainframe ecosystems
- 8-bit fixed-width representation
- Long legacy in enterprise computing

Disadvantages

- Not compatible with ASCII
- Limited use outside IBM legacy systems

Important correction

EBCDIC is an encoding family with several code pages; 'A = C1, B = C2...' is true for common EBCDIC variants such as IBM CP037, but should not be treated as a universal property of every EBCDIC code page.

Variable-Length Encoding

Definition

Different symbols may receive codewords of different lengths. It is used for less-storage capacity and its' efficiency.

Common techniques

Morse Code
- Shannon–Fano Coding
- Huffman Coding

In frequency-based compression, frequent symbols are generally assigned shorter codes and rare symbols longer codes.

Why Variable-Length Encoding?

Property	Fixed-length	Variable-length
Code length	Same for every symbol	Can differ by symbol
Capacity	Less	More
Efficiency	Less	More
Frequency used?	No	Yes, in frequency-based codes
Example	2 bits/symbol for 4 symbols	A=0, B=10, C=110, D=111

Short example

For 4 symbols, a fixed code needs 2 bits each.
If A occurs 5 times, B 3 times, C once and D once,
the variable-length example uses 5×1 + 3×2 + 1×3 + 1×3 = 17 bits.

But there is a condition

Variable-length codes must be designed so the decoder knows where one codeword ends. Huffman uses a prefix-free code: no codeword is the prefix of another.

Early Variable-Length Codes: Morse & Shannon–Fano

Morse Code

One of the earliest practical variable-length communication codes.
E = . T = − A = .− B = −...
Shorter patterns were useful for frequent letters.

Advantage: Mostly useful for Telegraphs,
Limitation: Human pause is needed to distinguish b/w two letters.

Shannon–Fano Coding

1. Sort symbols by decreasing frequency.
2. Split into two groups with nearly equal total frequency.
3. Assign 0 and 1 to the groups.
4. Recursively split each group.

Advantage: prefix-free code and frequency-aware.
Limitation: not always optimal.

Why Huffman next?

Shannon–Fano motivated systematic frequency-based coding, but Huffman's greedy construction guarantees an optimal binary prefix-free code for the given symbol frequencies.

What is Huffman Coding?

Definition

Huffman Coding is a greedy, lossless data-compression algorithm that constructs variable-length, prefix-free binary codes from symbol frequencies.

Core property	Meaning
Frequent symbols	Shorter codewords
Rare symbols	Longer codewords
Prefix-free	No codeword is the prefix of another
Goal	Minimize total / average code length among binary prefix codes

Greedy step

Repeatedly select the two least-frequency nodes, merge them into a new node whose frequency is their sum, and insert the new node back into a min-priority queue.

Example Input & Huffman Pseudocode

Example

Message M contains 100 characters:
A=5, B=9, C=12, D=13, E=16, F=45

Total frequency = 100.

Complexity

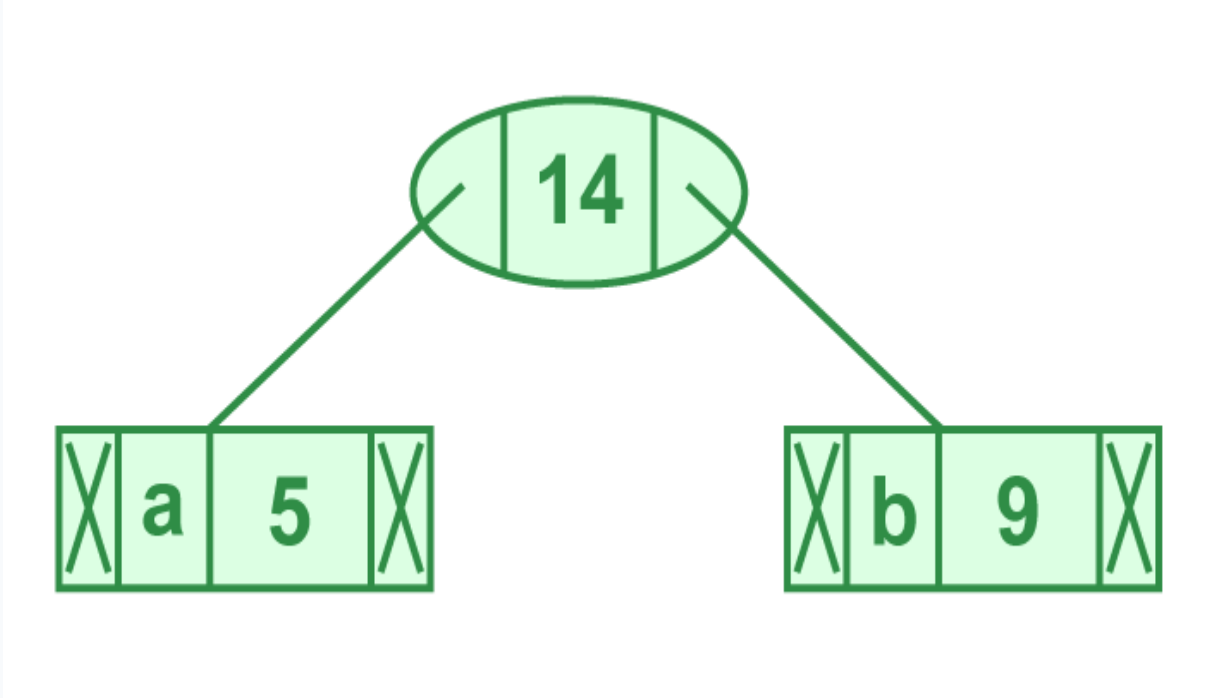
With a binary min-heap: build heap O(n);
n−1 iterations × O(log n) heap operations
⇒ total O(n log n). Tree storage: O(n).

```
HUFFMAN(C)
1  n = |C|
2  Q = C
3  for i = 1 to n – 1
4      z = new node
5      x = EXTRACT-MIN(Q)
6      y = EXTRACT-MIN(Q)
7      z.left = x
8      z.right = y
9      z.freq = x.freq + y.freq
10     INSERT(Q, z)
11 return EXTRACT-MIN(Q)
```

Huffman Tree — Step 1

A(5) + B(9) → 14

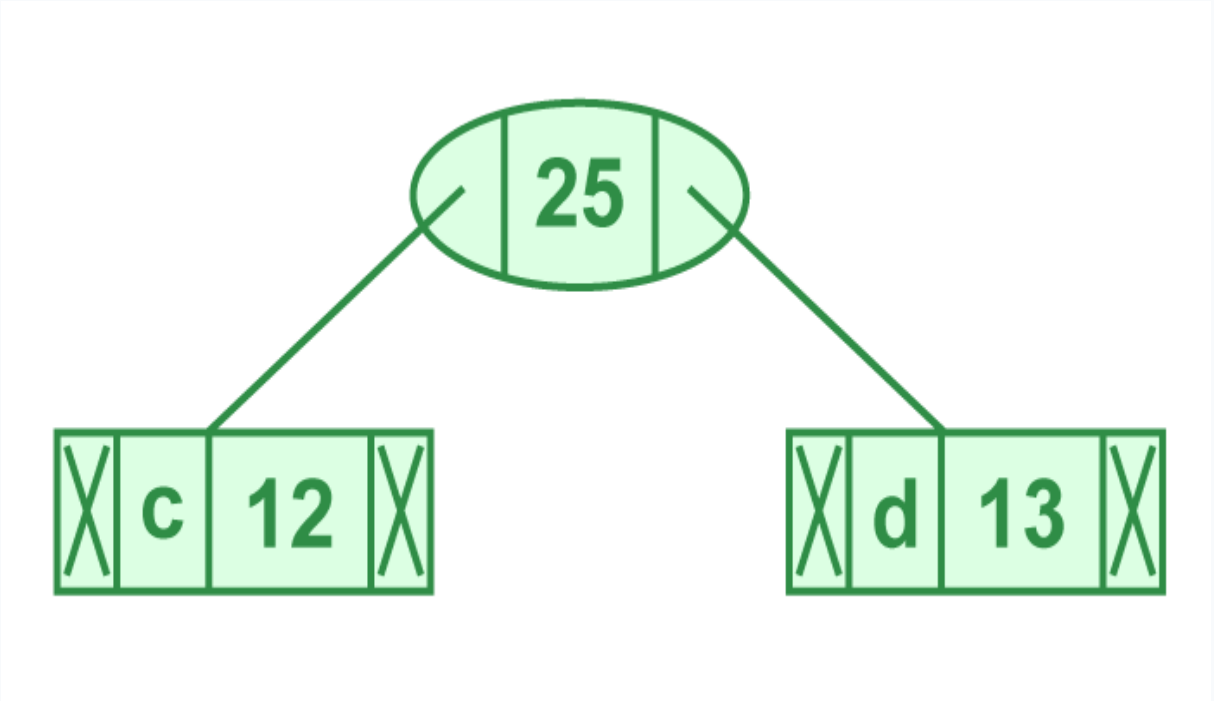
Initial heap: [5, 9, 12, 13, 16, 45]
Extract: 5, 9
Create: 14 = A+B
Updated heap: [12, 13, 14, 16, 45]



Huffman Tree — Step 2

C(12) + D(13) → 25

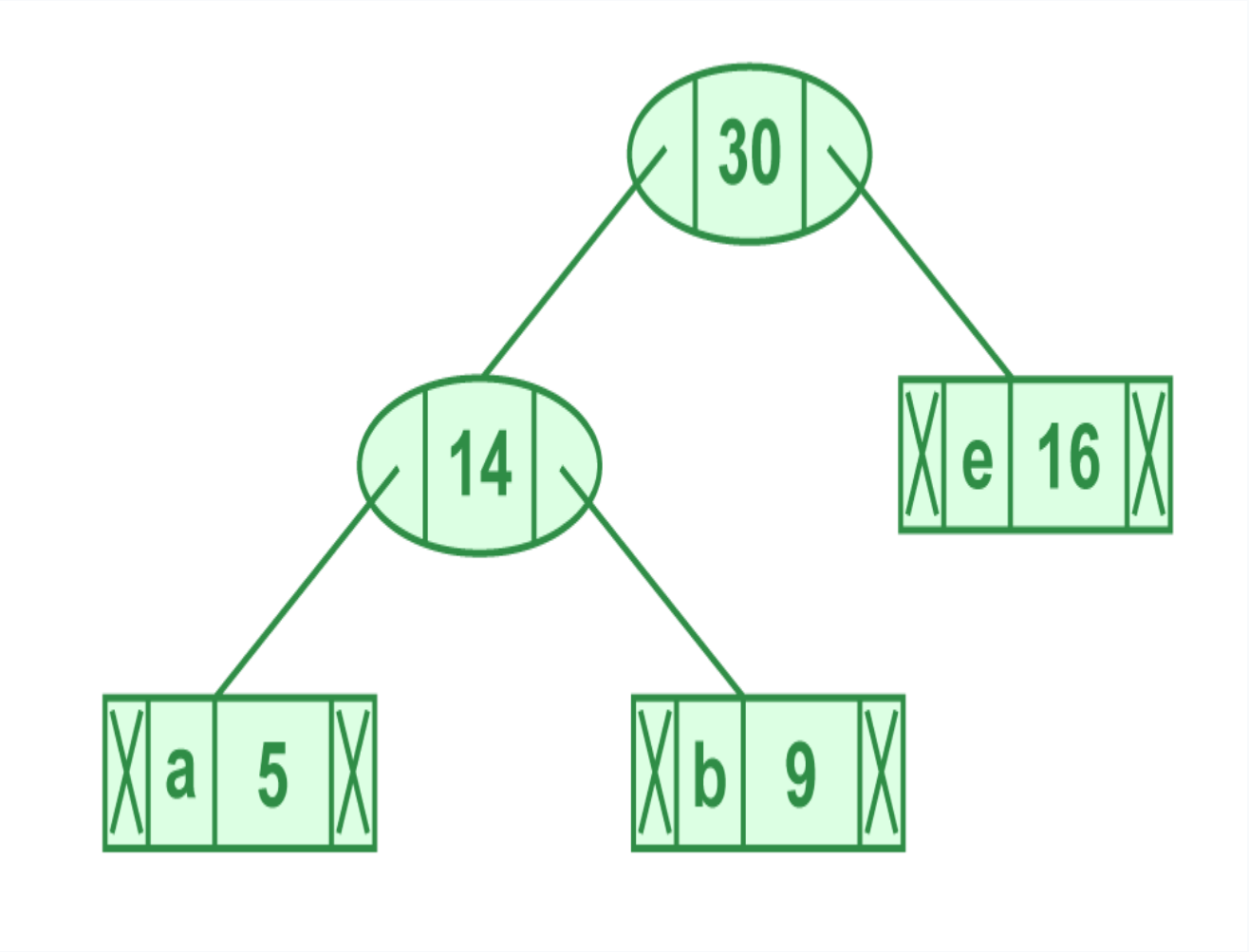
Extract: 12, 13
Create: 25 = C+D
Updated heap: [14, 16, 25, 45]



Huffman Tree — Step 3

14 + E(16) → 30

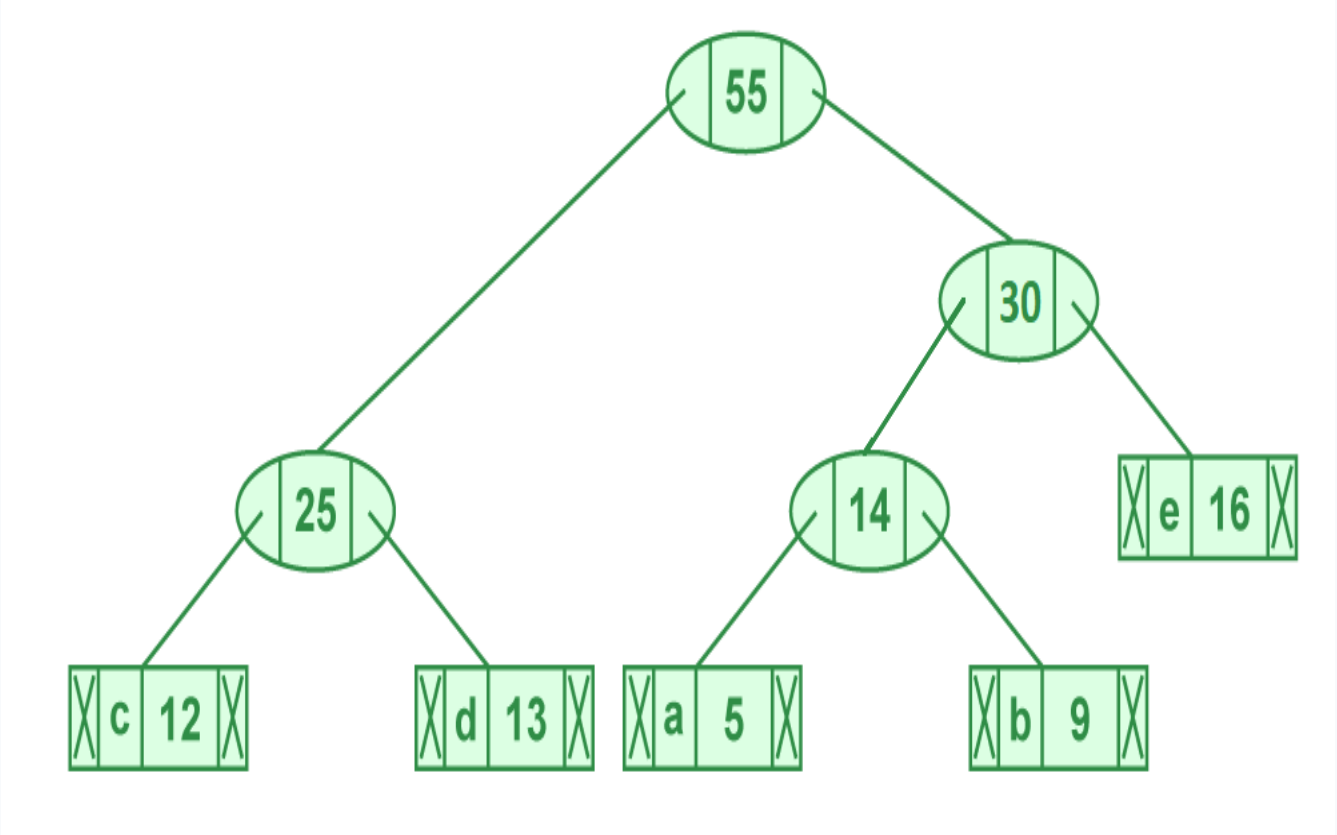
Extract: 14, 16
Create: 30 = (A+B)+E
Updated heap: [25, 30, 45]



Huffman Tree — Step 4

25 + 30 → 55

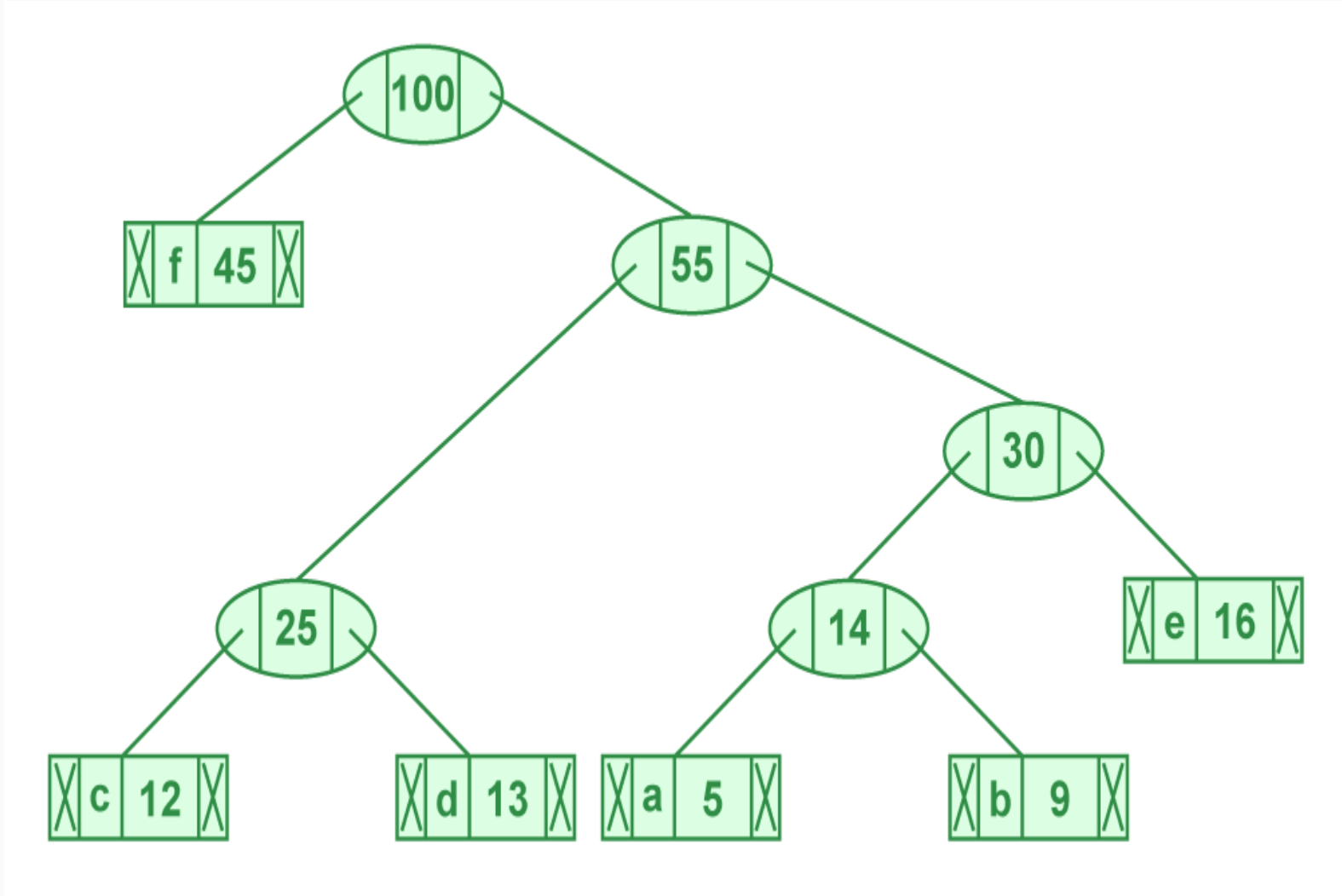
Extract: 25, 30
Create: 55 = (C+D)+(A+B+E)
Updated heap: [45, 55]



Huffman Tree — Step 5

F(45) + 55 → 100

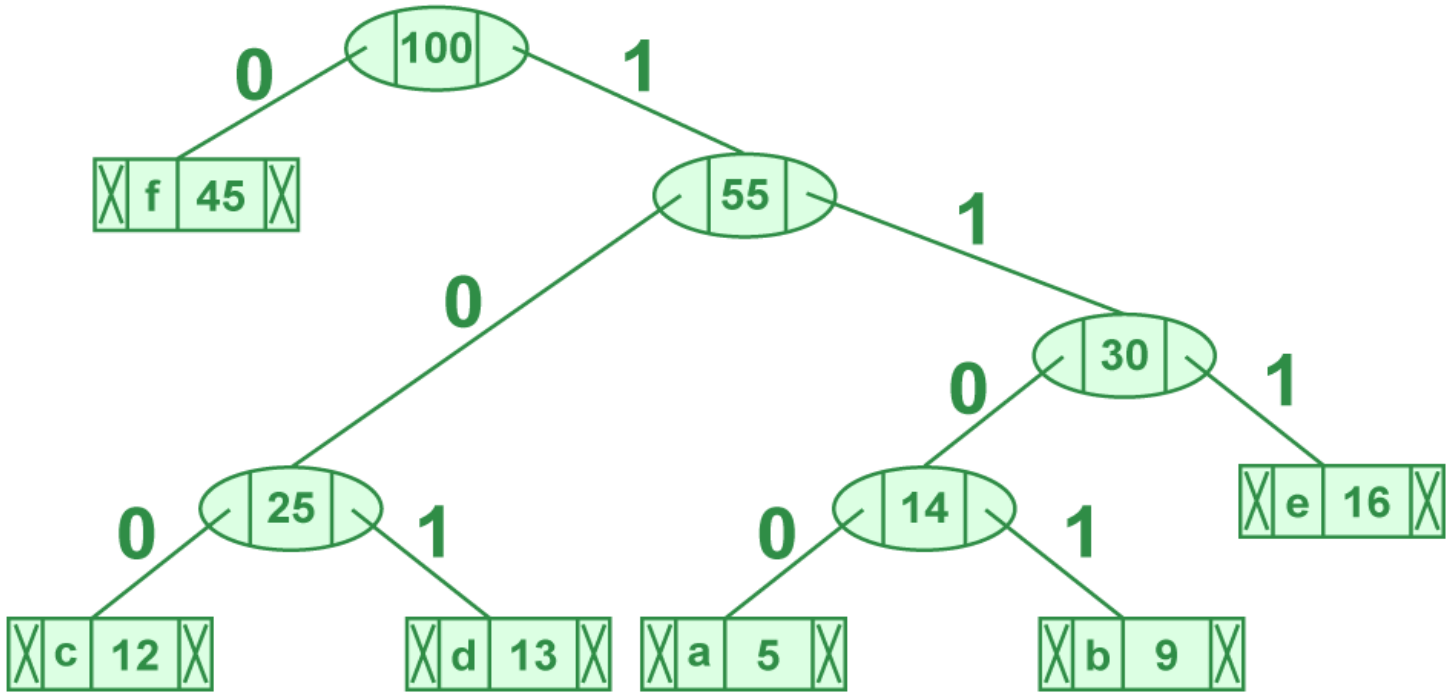
Extract: 45, 55
Create root: 100
Updated heap: [100] — algorithm terminates.



Full Huffman Tree with 0 and 1 Assignments

F(45) + 55 → 100

Extract: 45, 55
Create root: 100
Updated heap: [100] — algorithm terminates.



Rule

Left edge = 0 | Right edge = 1
The code for a symbol is the bit sequence along the path from root to its leaf.

Final Huffman Codes

Character	Frequency	Previous 3-bit code	Huffman code
A	5	000	1100
B	9	001	1101
C	12	010	100
D	13	011	101
E	16	100	111
F	45	101	0

Why the code lengths differ

F has the highest frequency (45), so it gets the shortest code (1 bit). A has frequency 5 and gets 4 bits. This trade-off reduces the weighted average code length.

Representation of Bits & Compression Gain

Character	Frequency	Huffman length	Bits used
A	5	4	20
B	9	4	36
C	12	3	36
D	13	3	39
E	16	3	48
F	45	1	45
Total	100		224

Fixed-length baseline

With 6 symbols, a fixed-length binary code needs 3 bits/symbol:
100 × 3 = 300 bits.

Huffman result

Huffman uses 224 bits for the 100-symbol message.
Saving = 300 − 224 = 76 bits
Reduction ≈ 25.3% (excluding tree/header overhead).

19. Encoding and Decoding

Encoding

Message: FACE
F = 0
A = 1100
C = 100
E = 111

Encoded stream:
01100100111

Decoding

Start at root and read one bit at a time.
0 → leaf F → output F → restart.
1100 → leaf A → output A → restart.
100 → leaf C → output C → restart.
111 → leaf E → output E → terminates.

Recovered message: FACE

Why prefix-free matters

Because no Huffman codeword is a prefix of another, the decoder can determine each symbol as soon as it reaches a leaf—no separators are required.

Advantages & Disadvantages

Advantages

- Lossless compression
- Optimal binary prefix code for the known symbol frequencies
- Frequent symbols receive shorter codes
- Greedy construction is conceptually simple
- Efficient implementation with a min-heap

Disadvantages

- Decoder needs the same codebook or tree
- Compression depends on the frequency distribution
- Little benefit when symbol frequencies are nearly uniform
- Tree/header overhead matters for small messages

Applications

File compression

Huffman coding is used in making of ZIP and GZIP file.

Transmission & storage

Fewer bits can reduce storage requirements and, when the coding overhead is small, reduce the number of bits that must be transmitted.

Complexity Analysis

Operation	Cost with binary min-heap
Build initial heap	O(n)
Extract-Min	O(log n)
Extract-Min (second node)	O(log n)
Insert merged node	O(log n)
Number of merges	n − 1
Overall	O(n log n)
Space	O(n)

Why O(n log n)?

There are n−1 merge iterations.
Each iteration performs heap operations whose cost is O(log n). Therefore:
(n−1) × O(log n) = O(n log n).

References & Further Reading

1. Cormen, Leiserson, Rivest & Stein, Introduction to Algorithms, 4th ed., MIT Press — Chapter 16.3, Huffman Codes.

2. David A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes,” Proceedings of the IRE, 40(9), 1952, pp. 1098–1101.

3. Jon Kleinberg & Éva Tardos, Algorithm Design — Chapter 4.8, Data Compression and Huffman Codes.

4. Princeton lecture slides: Data Compression / Huffman Coding — cs.princeton.edu/~wayne/kleinberg-tardos/pearson/04Huffman-2x2.pdf

5. University of Maryland CMSC 351 Huffman Encoding notes — math.umd.edu/~immortal/CMSC351/notes/huffman.pdf

6. McMaster University Huffman Codes notes — ece.mcmaster.ca/~xwu/703/huffman.pdf

7. David Salomon, Data Compression: The Complete Reference, 4th ed. — prefix codes, Huffman, arithmetic and universal codes.

8. Huffman Coding Algorithm – www.geeksforgeeks.org

Thank You